

Chapter 4: Rule-Based and Model-Based OPC

Let's review key concepts we have learned so far.

In a patterning process, shapes on a photomask are transferred to wafer, i.e., printed on silicon, through exposure of photoresist to an optical system, i.e., a stepper, followed by development and etching.

Shapes on design layout are the targets to print on silicon. Usually, they are Manhattan polygons, or at most, polygons containing 45-degree edges, with only a few vertices. However, if such shapes are drawn on photomask, their printing on silicon would be distorted due to limitation of equipment and materials. Some of the limitations are better understood and quantified than others, based on principles of physical optics and photochemistry.

Enter OPC, ILT, and other Reticle Enhancement Technologies (RET). These technologies make corrections on mask (also known as reticle). The corrections are derived from physics/chemistry theories and empirical data, and aim to eliminate/reduce the distortion. RET is conducted via computer algorithms which take in a design layout and output a mask layout, with shapes much more complicated than the targets.

OPC in Chip Manufacturing

Among all RET tools, OPC is the most mature, most developed, most practical, and therefore most widely used. Sometimes OPC is synonymous to RET.

However, the name OPC is somewhat misleading. Mask corrections through OPC are based not only on optical exposure but also other patterning steps, i.e., development and etch, or even none patterning steps, such as CMP; nor is OPC only for proximity (local) effects, for example, EUV models contain long-range flare.

Before we dive deep into technical details of OPC, let's take a high-level review of how OPC is employed in chip manufacturing.

Technology Node and Device Layer: A chip contains many device layers. Each layer takes one mask to fabricate (multiple-patterning-process would require more than one mask). Among these layers are “critical” ones, which contain smallest and closely packed shapes allowed by the technology node of the chip. For example, on a chip of 14nm node, the narrowest lines would be 14nm wide; the tightest array would be 14nm lines with 14nm spaces.

Out of the many device layers only a few are critical, i.e., poly, contact, some metals. OPC, especially aggressive OPC, would be required for these layers. Non-critical layers, such as implant, contain much bigger shapes and would require no OPC or simple OPC.

OPC in Chip Manufacturing (continued)

Fab: Process-of-Record for patterning is intellectual property of a fab. Equipments, materials, process flow and parameters, etc., are unique to a fab. Models and rules in OPC are derived from process parameters and empirical measurements with the process. Therefore, even for same technode and same layer, each fab would need to develop its own OPC.

EDA Platform: Last but not least, as we shall see, OPC (and other RET technologies) are developed upon platforms of EDA companies, for example, Calibre of Mentor Graphics, Proteus of Synopsys. Each fab chooses one EDA company for their OPC work, sometimes even different EDA company for different node/layer. These platforms each have proprietary approach and algorithms.

In summary: OPC is specific to technology node, design layer, fab, and EDA platform.

OPC 101

K1/K2 factor: OPC aims to improve quality of printing on silicon. The quality is often described by two equations:

Resolution $CD = k_1 * (\lambda/NA)$. k_1 ranges from 0 to 1.

Depth of Focus $DOF = k_2 * (\lambda/NA^2)$. k_2 ranges from 0.4 to 1.

Here, CD can be defined as, at the best exposure/focus condition of a stepper, the minimum size of printable isolated shape or half of the minimum pitch of a resolvable array. DOF is defined as the defocused plane on which the phase error between it and the best focus plane is 90 degree. A production environment requires certain defocus budget. DOF needs to be big enough to accommodate that.

Smaller CD and bigger DOF are desired. While λ and NA are fixed parameters of stepper, OPC can decrease k_1 and increase k_2 .

OPC 101 (continued)

So, how does OPC decrease k_1 and increase k_2 ? As time goes, OPC becomes ever more sophisticated:

- Add serif, anti-serif and hammer head to corners and line ends.
- Add assist features which themselves do not print but can better target feature printing.
- Move edges based on rules or simulations.
- Tile low density area with dummy shapes.
- Use phase shift mask: attenuate or alternate.
- Decompose a layout into multiple masks.
- Locate certain hotspots by pattern matching for special handling.
- Apply long range corrections for EUV patterning.
- Allow curvilinear shapes.

OPC 101

All the above OPC approaches fall into three categories: rule-based, model-based, and pattern-based, which is a more recent development.

Rule-Based: Corrections need a recipe.

Model-Based: Corrections need a model and a recipe.

Pattern-Based: Corrections need a pattern library and a recipe.

Here, recipe is a common jargon for a computer algorithm, with which an OPC job reads in a design (in gds/oasis format), runs on CPU/GPU, w/wo (w or w/o?) DP, and writes out a mask layer, again, in gds/oasis format.

Rule-Based OPC

In a typical, conventional definition, rules in the context of rule-based OPC can be described as: a) how to move edges of a polygon and b) how to add sub-resolution assist features, according to numerical and logical values of intra- and inter- polygons on a design layout. Here are some examples of the numerical/logical values.

Numerical Values: area; topology (w or w/o holes); count of vertices; aspect ratio of rectangles; direction/length of an edge; degree of convex/concave corner; density; space between edges/corners.

Logical Values: inside/outside, overlap, touching, interacting, Booleans, etc., of edges and polygons.

In developing a recipe for a device layer, say metal, the recipe can use polygons from metal and additional layers. For example, a rule may be “how much a metal line end encloses a contact.”

Based on empirical data, a recipe can be developed. Examples are in next slides.

In a broader definition, rules in the context of rule-based OPC can also include: a) how to “color” regions with different phases (in PSM); b) how to add dummy patterns to low density region (tiling) ; c) how to decompose a design layout into multiple masks (decompose)--since all these operations do not need a model.

A Sample Script

Here is a sample code of Calibre OPC recipe, in which we shall see concepts about layout, i.e., precision, top cell, etc.

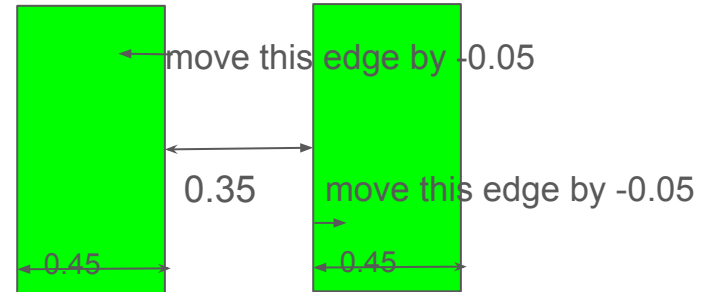
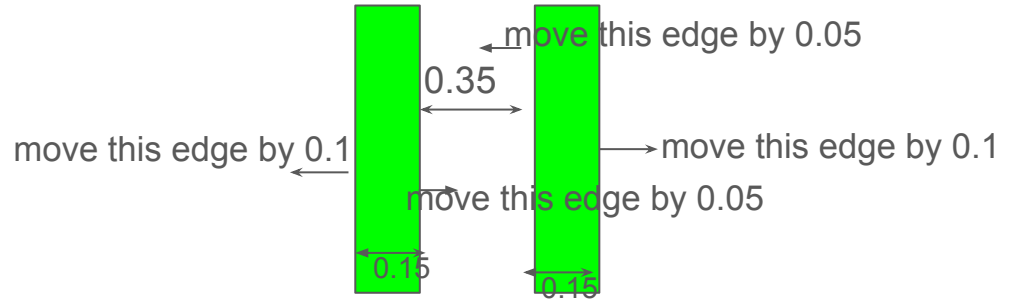
The rule uses a “size” command, which means move all edges of a polygon by same amount, positive for outward, negative for inward. It applies the size command only on polygons with area ≤ 1.0

```
LAYOUT PATH mytest.gds #define input design gds
PRECISION 1000 #define layout grid
LAYOUT PRIMARY mytest #define topcell
LAYOUT SYSTEM GDSII
DRC RESULT DATABASE mytest_out.gds # define output mask gds
# main body of the correction
LAYER MAP 1 DATATYPE 0 100
LAYER metal 100
small_metal = AREA metal <= 1.0
metal_sized = SIZE small_metal BY 0.005
DRC CHECK MAP metal_sized 2 0 #define post-opc mask layer/datatype in the output gds
```

Examples of Edge-Moving Rules

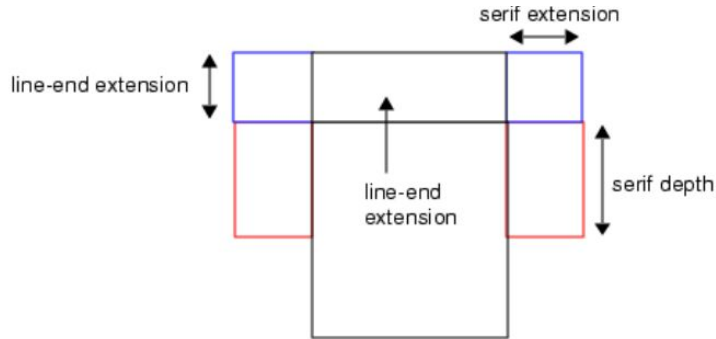
Example 1: Move edges inward (negative) / outward (positive) based on width/space of an array of lines to correct thru-pitch variation.

	Line Width					
	≤ 0.1	$>0.1 \leq 0.2$	$>0.2 \leq 0.3$	$>0.3 \leq 0.4$	$>0.4 \leq 0.5$	>0.5
0.0	0	0	0	0	0	0
$>0 \leq 0.1$	0	0	0	0	-0.10	-0.15
$>0.1 \leq 0.2$	0	0	0	-0.05	-0.10	-0.10
$>0.2 \leq 0.3$	0	0	0	-0.05	-0.05	-0.10
$>0.3 \leq 0.4$	0	0.05	0	0	-0.05	-0.05
$>0.4 \leq 0.5$	0	0.10	0.05	0.05	0	0
$>0.5 \leq 0.6$	0	0.10	0.05	0.05	0	0.05
>0.6	0	0.10	0.10	0.05	0	0.05

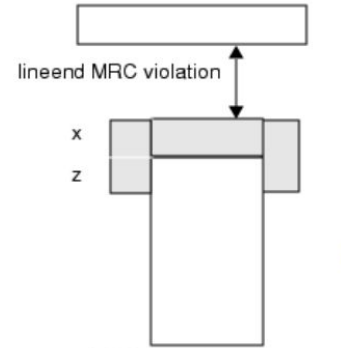


Examples Edge-Moving Rules (continued)

Example 2: Add serif to correct line-end shortening



```
x {OPCLINEEND POLY
  //default rule
  WIDTH <0.3 HEIGHT >0.2          END 0.02  SERIF 0.002 0.1
  //correction rule 1
  WIDTH <0.3 HEIGHT >0.2 SPACE <=0.3  END 0.005 SERIF 0    0.115
  //correction rule 2
  WIDTH <0.3 HEIGHT >0.2 SPACE >0.3 <=0.35 END 0.01  SERIF 0.01 0.11
}
```

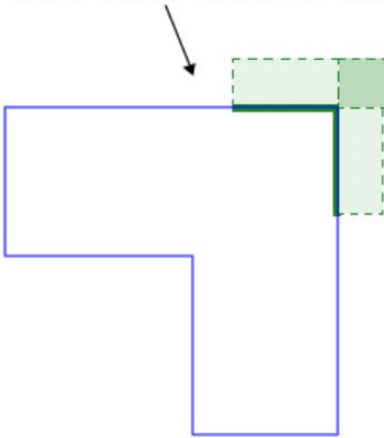


Initial correction:
END = x
SERIF depth = z
Total SERIF height = x + z

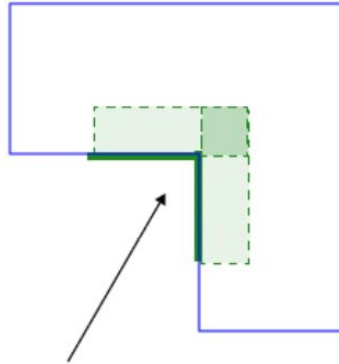
Examples of Edge-Moving Rules (continued)

Example 3: Add serif /anti-serif to correct to convex/concave corner rounding.

EXPAND EDGE X OUTSIDE BY 0.05 CORNER FILL



EXPAND EDGE M INSIDE BY 0.05 CORNER FILL



original polygon



convex edge ==2
// also convex edge > 1

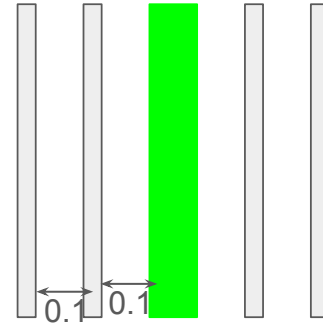
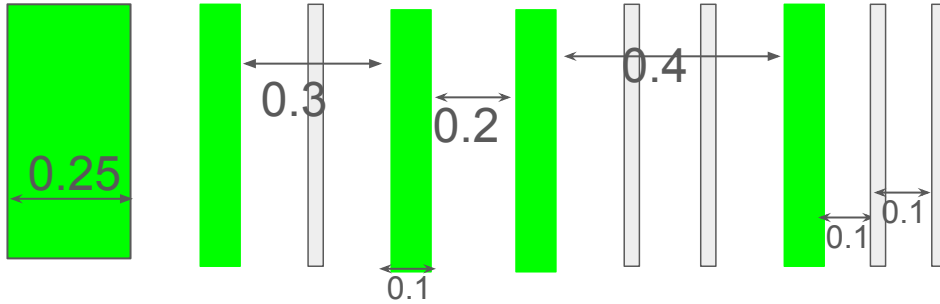


convex edge ==1
//also convex edge < 2

Examples of Assist Features Rules

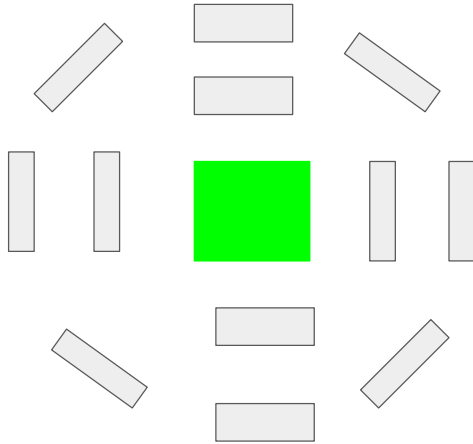
Example 1: Add assistant features (AF) for 1D array of main features (MF)

- $0.1 \leq \text{width} \leq 0.2$; $0.3 \leq \text{space} < 0.7$
 - AF width = 0.05; min space between AF = 0.1; min space between AF and MF = 0.1; If # of AF > 1, AF should be evenly distributed between MF
- $0.1 \leq \text{width} \leq 0.2$; 0.7 space (an isolated edge)
 - AF width = 0.05, space between AF = 0.1; space between AF and MF = 0.1; two AF added to each side of the MF



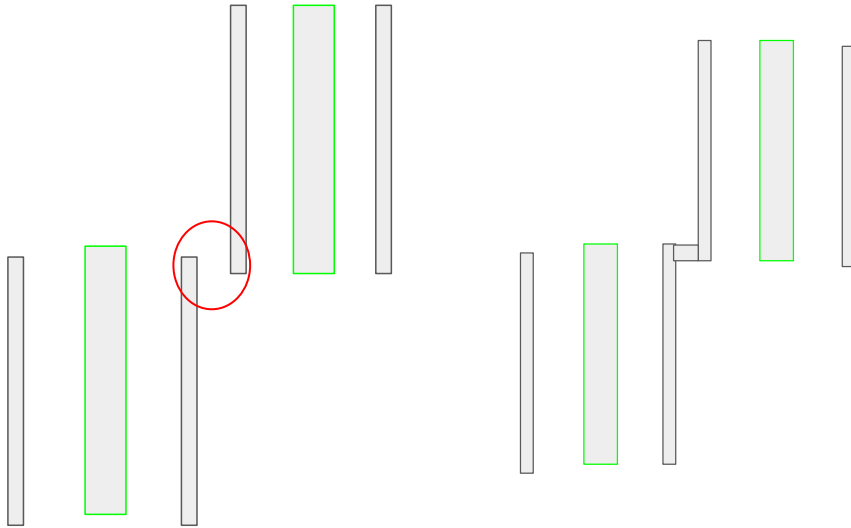
Examples of Assist Feature Rules (continued)

Example 2: Add assist features around a 2D square



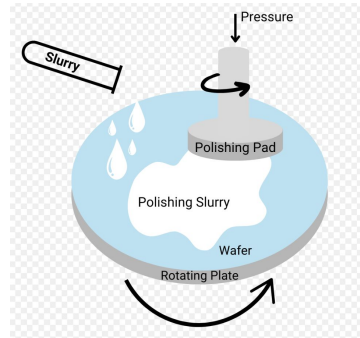
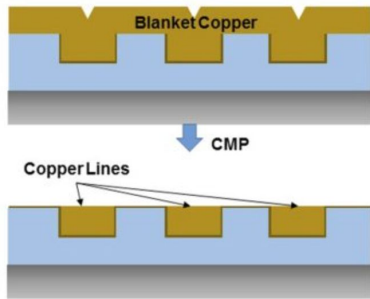
Examples of Assist Features Rules (continued)

Example 3: In the circled region the space between two AF is too small to resolve. The two AF are connected to resolve the conflict.



Tiling and Chemical Mechanical Planarization (CMP)

Tiling is a method to enhance CMP. CMP is a process to polish a layer of materials, for example, metal, in order to flatten the surface across the whole wafer, because uneven surface causes defocus in exposure and therefore hurts quality of image. But CMP polishing rate varies with pattern density. Tiling can reduce pattern density variation by adding dummy patterns in empty area.



Dummy Patterns

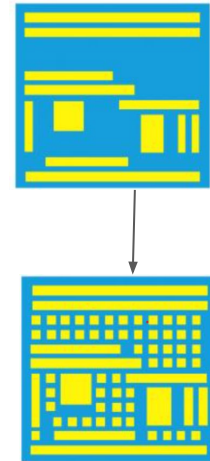
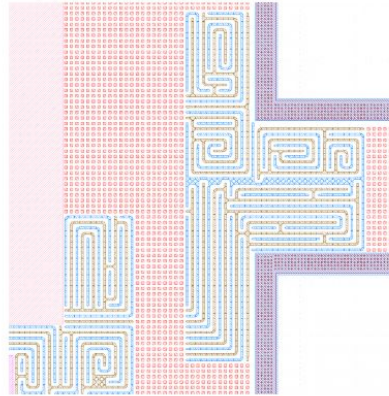
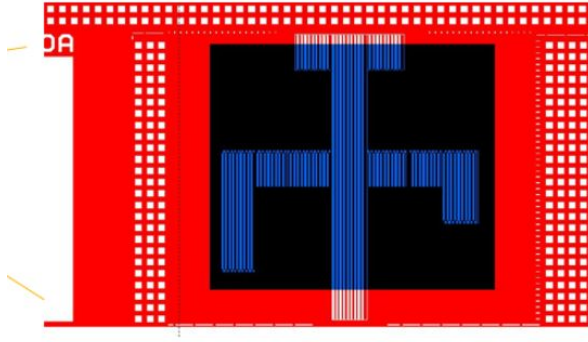
Unlike sub-resolution assist features, dummy patterns must print on silicon. However, these dummy patterns are totally isolated from circuits and therefore unfunctional.

Most tiling approaches are rule-based. The rules define size of dummy patterns (typically, squares/rectangles) and their arrangement based on target density. Dummy patterns must be kept at a “safe” distance from circuits to avoid introducing parasitic capacitance that would affect timing.

Cutting edge chip making may also employ model-based tiling, in which thickness variation is simulated with models of CMP and tiles are added to density “hotspots” predicted by simulations.

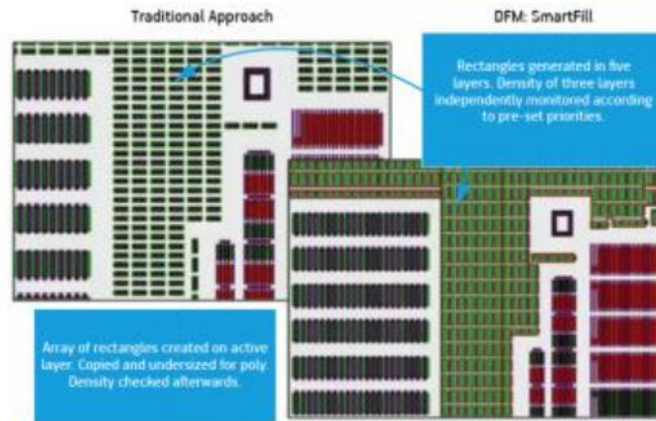
Traditional Tiling Method

A grid of identical tiles are dropped into empty spaces over the entire chip area. Size of the tiles and spaces between them are pre-determined based on target density. Such tiling are straightforward but can not account for local variations of density of existing patterns. Therefore it may lead to over filing in some regions.



Smart Tiling

Recently, sophisticated, smart tiling has become mainstream. With such approaches, a chip is scanned window-by-window. Within each window, local density of the chip is calculated, and various shapes are dropped in to bring the density up to the target.



Dummy Fill on left, SmartFill on right. Source: AMD

Phase Shift Mask (PSM)

PSM was adopted for 180nm~130nm nodes and has since become industrial standard. Although several flavors of PSM have been developed, only two have been proved to be production worthy: alternative PSM and attenuated PSM.

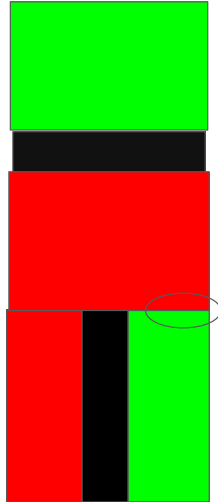
Alternative PSM can enhance printing image better, but it requires two masks/two exposures to make one layer and hence doubles process time. Therefore, attenuated PSM has become the mainstream.

Why Alternate PSM needs two masks/two exposures for a layer?

Phase Assignment Conflict

Black = Chrome, Opaque
Green = Quartz; Transparent; Phase = 0
Red = Quartz; Transparent; Phase = 180

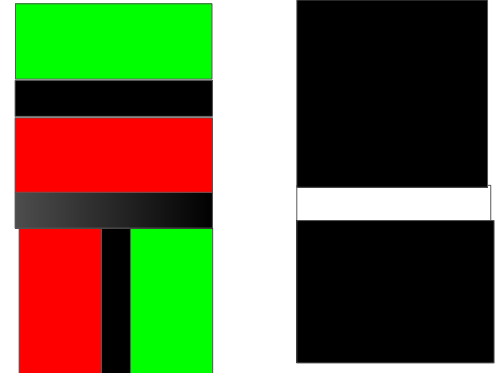
Phase Conflict: Black is the target, but light destruction in the circled area will introduce a line—an extra feature—printed on silicon.



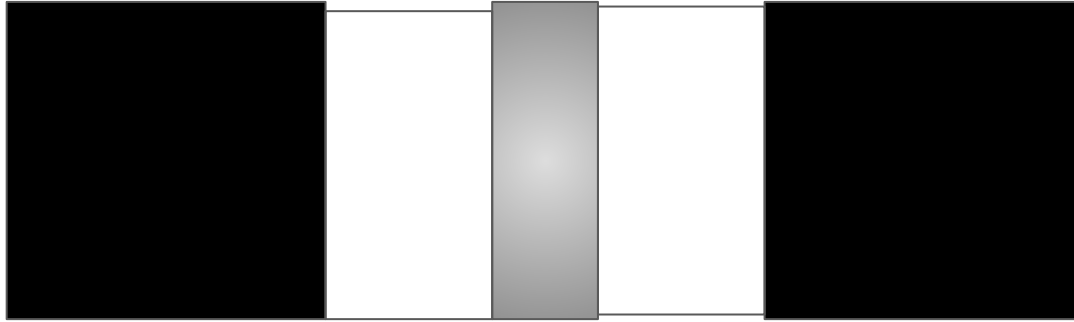
Phase Assignment Conflict Solved by 1st exposure with a PSM followed by 2nd exposure with a binary mask

1st exposure: Three black areas will be printed on photoresist.

2nd exposure: The middle black area which is NOT the target is exposed and cleared. The other two black areas, which are target, will be protected.



Why attenuated PSM becomes mainstream?



Black = Chrome; Opaque

White = Quartz; Transparent; Phase = 0

Shade = MoSi; 6% transparent, phase = 180

Shade area is a critical feature where attenuated PSM enhances printing.

Black area are non-critical feature which do not need the help of PSM.

Multiple Patterning

Alternative PSM did not become mainstream because it doubles mask/exposure. However, other multiple patterning processes do become indispensable for the most advanced technology nodes— especially, if EUV litho is not available—in spite of the many challenges they bring about to process, cost, and time.

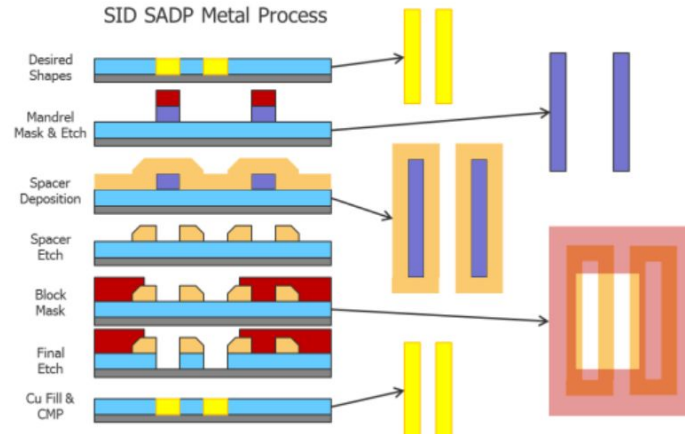
There are various ways to create multiple masks to one device layer. Overall, they can be characterized as “splitting” or “cutting.”

Splitting: Splitting a layout is to separate shapes/parts-of-shapes into two layouts.

- Pitch splitting: for parallel lines too close to each other, i.e., for sub-resolution pitch, group every alternate lines. Within each group, the pitch doubles.
- Orientation splitting: group vertical lines together and horizontal lines together. Dipole illuminations can help printing each group, respectively.
- Device splitting: Patterns inside memory circuits are dense and repetitive, while inside logic circuits sparse and random. Separate memory circuits and logic circuits into two exposures in which the process conditions are optimized, respectively.

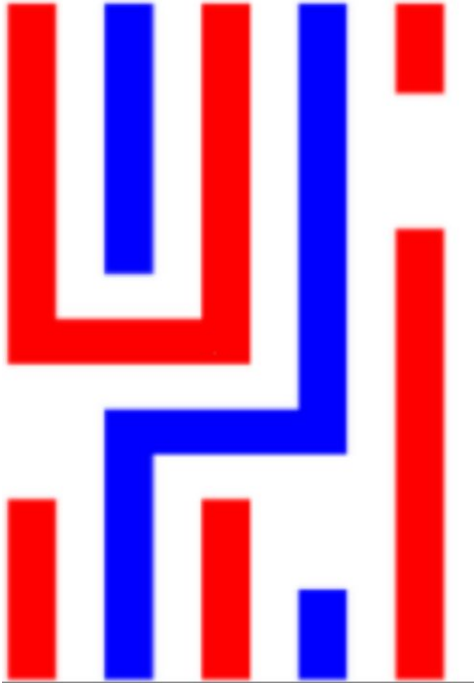
Multiple Patterning (continued)

Cutting: Sometimes, printing arrays of lines can be optimized in process; but simultaneously opening up small spaces, such as tip-to-tip line ends, holes, etc., would be difficult. An option is to cover the spaces in the 1st exposure and then use the 2nd exposure to cut open the small spaces. Another usage of cutting is in Self-Aligned-Double-Patterning. As shown in this example, a layer of mandrel, which is a temporary material, is deposited on substrate; mandrel is patterned in 1st exposure/etch; another temporary materials is then deposited which will react with the mandrel to form spacer around the edge of mandrel; after etching, rings of spacer remains, 2nd exposure cuts off the rings so only the target metal lines are flanked by spacers; then the metal lines are formed by Cu fill and spacers are removed.

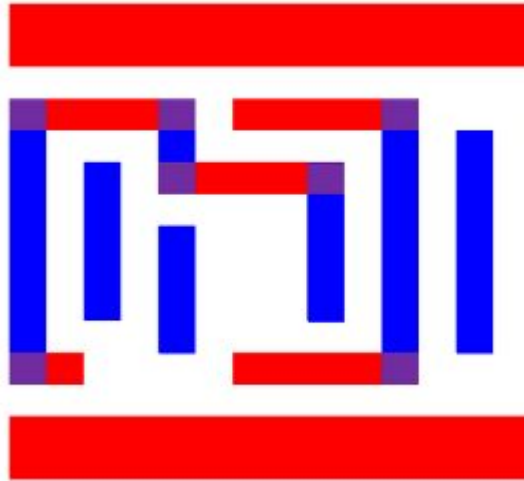


Example of Multiple Patterning

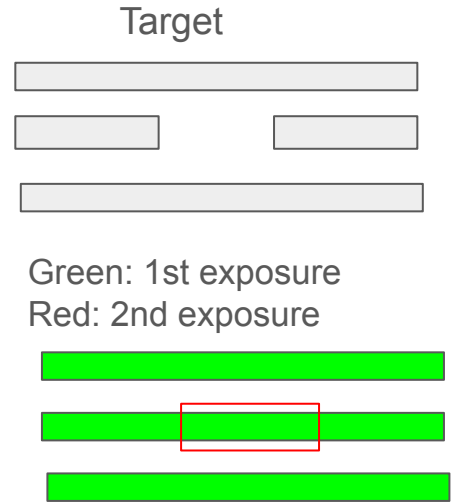
Pitch Splitting



Orientation Splitting



Cutting Tip-To-Tip Space



OPC: From Rule-Based to Model-Based

In Rule-based OPC (per conventional definition), a predefined table contains rules that capture certain geometrical/logical characters of layouts. These characters intend to cover the proximity effects in patterning. Edge moving and AF creation are based on these characters.

As layout becomes ever more complicated, accurate capturing of geometrical/logical variations requires too many rules that are difficult to define. It is like the US tax code, which forever expands with time due to new demographic and economic situations. Yet however comprehensive the tax code aims to become, people always find loopholes and take advantage of them. Rules are powerful and efficient for making initial guesses in layout correction. But they are too coarse for modern technology nodes.

In Chapter 3, we described litho models. Like pilots on fly simulator, litho simulations with such model produce virtual wafer images of mask patterns. With such virtual image, layout corrections can be based on deviations between such image and target design pattern. That is Model-Based OPC.

Model-Based OPC

A model-based OPC algorithm has two components: model and recipe.

Model: Model dictates accuracy of the virtual wafer image. However, more accurate models often consume more computation. Balance must be stricken between accuracy and computer work. Usually, an ADI model at nominal conditions followed by an AEI model are deployed to simulate wafer image.

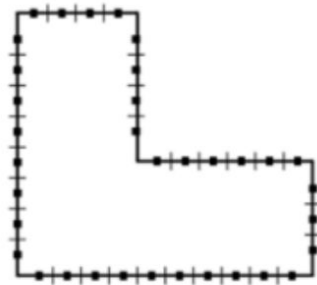
Recipe: Recipe translates RET engineer's procedure and methodology for layout correction into computer algorithms. RET engineers develop a recipe to sample layout, compute image-target deviation, move edge, and minimize repetitive work. Major components of a recipe include:

Segmentation; Edge Placement Error (EPE) Calculation; Corrections; Mask Rule Check (MRC); Hierarchy Management.

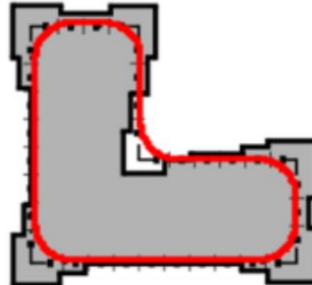
Segmentation

A straight edge on a polygon from design layout are comprised of portions, i.e., segments, which, from patterning point of view, have various features. For example, for a segment near a corner, printing tends to round more than a segment away from the corner. Therefore, these two segments should be corrected differently. Segmentation, also called dissection, is the fundamental steps of OPC. This step slice a straight edge based on rules that capture features. On each segment an evaluation site is assigned, the simplest way is to assign the evaluation site to the dead center of the segment. Common segmentation rules are illustrated on next page. Usually OPC recipe defines segmentation parameters such as max/min segment length, corner/line end segment length, evaluation site, etc., while many other parameters/procedures are built-in so that the recipe would not be too complicated.

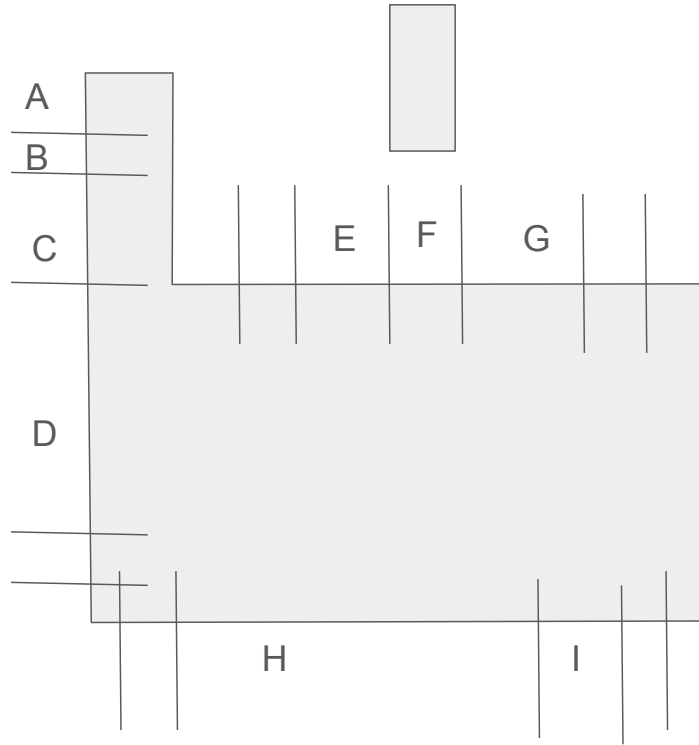
Segmented Pattern



Polygon Mask



Common Segmentation Rules



A: corner segment

B: one or two segment(s) for rippling effect of a corner

C: segment created to line up with the edge facing it.

D: segment connecting C to another corner

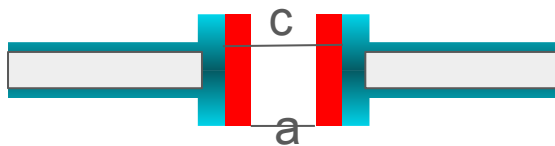
E/F/G: segments created due to influence of nearby shapes.

H/I: a long segment broken into two in order not to violate maximum segment length.

However, if I is shorter than minimum segment length, it will be merged with H or the segment on the right.

Model-Based OPC (continued)

- **Edge Placement Error (EPE) Calculation:** EPE is measured as the distance from the edge of the target, at normal direction, to the simulated image, at the evaluation point of a segment.
- **Corrections:** Based on the EPE, a segment is moved inward/outward, in steps, to reduce the EPE. Corrections are done in iterations. In each iteration, every segment is evaluated and corrected. The step size, maximum number of iterations, and tolerance of EPE are pre-defined. If the tolerance has been reached in less than the maximum number of iterations, the correction would stop.
- **Mask Rule Check (MRC):** While moving the segments, mask making rules must be obeyed. Typical mask making rules are minimum space, minimum width, etc. Here is an example:



Grey: target

Green/Red combined: mask correction needed to bring EPE within tolerance

C: minimum space required by MRC, which is $> a$.

Green: final mask correction after MRC

Model-Based OPC (continued)

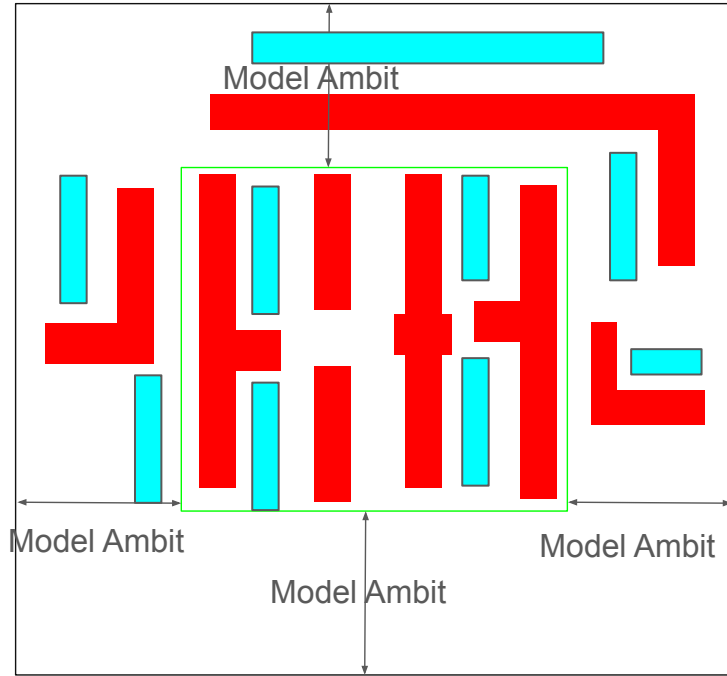
Hierarchy Management: As discussed in Chapter 2, design layout is usually highly hierarchical. However, in OPC, a cell in the design layout can be corrected in various ways depending on the **context** of its placement, typically within 1 ~ 2 μm . If the context is same, correction of cells can be reused. This is especially important for correction of memory array, where cells away from the array boundary have same context, which are different from the context for cells near the boundary.

In simulation and correction, a layout is partitioned into **templates**. Each template is processed individually. Multiple templates can be parallelly computed. Templates with identical layout counts only once.

Each template is a square, i.e., 5 μm by 5 μm . Centered in the square is the **Main Zone**, a square within which shapes are corrected by OPC. The **Main Zone** is surrounded by a border **Context Zone**. Shapes inside the **Context Zone** are used in simulations to generate images of the **Main Zone**, but they are **not** corrected. Shapes inside the **Context Zone** are within the proximity of shapes in the **Main Zone**. The range of the proximity, hence the extent of the **Context Zone**, is defined by the OPC **Model Ambit**.

Model Ambit is the cutoff distance between shapes, above which two shapes apart will not be included in the convolution of the model with shapes. It is a parameter of model kernel.

Template Illustration



Black: Template boundary
Green: Boundary of Main Zone. Within the Main Zone AF (blue) will be included in simulation, but corrections only applies to design features (red).
Between Green and Black: Context Zone, within which all shapes are included in simulations to create image within the Main Zone, but none gets corrected.

Hierarchical Management in Simulation (continued)

Single-Cell Correction/Insertion: A memory array can contain numerous instances of a cell. In the design layout, those cells away from the boundary of the array have SAME contexts. Therefore, a template should be constructed which has ONE cell in the Main Zone, and a few others in the Context Zone. For example, a 3x3, or 5x5, cluster. **Correction** is done to the center cell of the cluster. The corrected cell can then be **inserted** to all other cells EXCEPT those at the boundary of the array. Similarly, a template for the corner cell, the edge cell, etc., could be created. This is the basic idea of taking advantage of design hierarchy in OPC and maintain a hierarchical structure in the post-out layout.

Template Creation for Logics: Design layout are random and non-periodic. Templates are seldom identical. Template size is usually dictated by the **Field Size** allowed by simulation engine. **Field Size** is the maximum window in space domain that simulation engine is capable of processing as a unit. Field size can be defined as Field Matrix Size times Field Sampling Space. It is a constraint due to system memory limit and other factors. Template size CANNOT exceed field size.

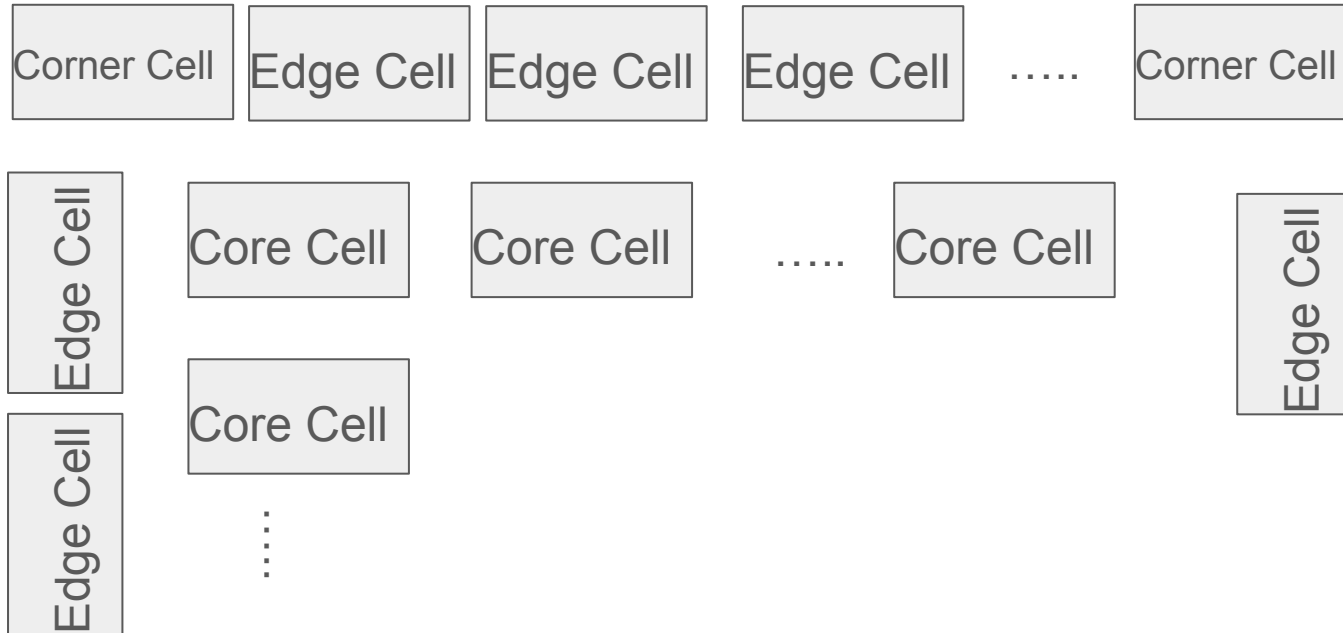
However, if standard library cells, such as NAND, have same contexts, the above single-cell correction/insertion method can also apply.

If template boundary cuts through a shape, it is important to ensure that after correction, the shape is “stitched together.” There are methods to re-route template boundary so it goes around, rather than cutting through, shapes. (Ref)

Pattern Hashing: To create unique templates of certain size, OPC engine scans a layout with proprietary geometrical hashing functions which generates a numeric **hash key** for each template. Templates with identical hash index are identical geometrically. Therefore, only templates with unique hash key need to be simulated/corrected.

Illustration of Memory Array Single-Cell Correction/Insertion

Core cell, edge cell, corner cell, each needs to be corrected just once.



Challenge and Limitation of Model-Based OPC

As mentioned before, model-based OPC does not involve predefined rule tables as in rule-based OPC. However, model-based OPC still utilizes rules for segmentation and evaluation. Also, model-based OPC usually restricts edge moving in the normal direction only.

Cutting-edge technology nodes call for more aggressive and flexible mask corrections. Among them is the concept of “free-form mask,” i.e., treating a mask as continuous, grayscale pixels and allowing the mask shapes to vary pixel by pixel. Inverse Lithography Technology has been developed based on such concept.

Chapter 4: Summary

In this chapter, we have discussed OPC: a generic term for creating mask shapes with computer algorithms to enhance the quality of printing target shapes of design layout.

In a conventional definition, an OPC algorithm usually flows this way: rule-based filtering of no-OPC patterns (including dummy patterns, some test features, metrology features, text labels, etc.), rule-based AF insertion, and, finally, edge moving based on either rules or simulations.

Rule-based operations run fast, but are usually on flattened database and without parallel processing. Simulations are computationally expensive. To make simulations efficient enough at production level, it is important to deploy hierarchical management of layout and massive parallel processing.

In a broader definition, OPC also includes tiling and mask decomposition. These techniques do not involve edge moving, but they do introduce/manipulate mask shapes.

The mask layout through OPC should be verified by MRC and litho-based rule-checking. The verification step will be discussed in the next chapter.

Algorithms of OPC and other RET technologies are built on EDA platforms. These platforms are completely incompatible with each other. A fab may employ different EDA platforms for RET solutions of different technology nodes or even device layer.